

MODULE 10

Agentic AI with Microsoft Services

From AI assistance to goal-driven autonomous workflows.

Agentic AI
concepts

Copilot as an
agent

Microsoft AI
integration

Tools and
function calling

Autonomous
workflow
design

API
orchestration

Agentic
microservices

Security and
observability

LEARNING OBJECTIVES

Agents, tools, orchestration, Microsoft services and microservices use cases.



BY THE END OF THIS MODULE, YOU CAN

1. Explain agentic AI
2. Distinguish assistants, copilots and agents
3. Identify core agent components
4. Explain an agent execution loop
5. Describe Microsoft agent services
6. Integrate agents with enterprise APIs
7. Design autonomous workflows
8. Use tool / function calling
9. Design single- and multi-agent solutions
10. Orchestrate Spring Boot microservices
11. Apply agent security guardrails
12. Monitor agent workflows

THE GOAL Controlled autonomy around clearly defined enterprise capabilities.



FIVE SECTIONS

1

Agent fundamentals

Goal → Reason → Act → Observe

2

Microsoft agent ecosystem

Copilot Studio · Foundry · Models · Tools · Knowledge · Workflows

3

Workflow design

Goal → Plan → Tools → Conditions → Completion

4

Microservice orchestration

Agent → Order · Payment · Inventory · Shipping APIs

5

Enterprise controls

Identity · Authorization · Guardrails · Approval · Audit · Observability

KEY TAKEAWAY Learn to give agents controlled autonomy over well-defined enterprise capabilities.

FROM MODULE 9 TO MODULE 10

AI helped us write the code — now AI takes actions on the running services.

MODULES 1–9 GAVE US

- ✓ Order, Payment, Inventory and Shipping APIs
- ✓ Events, idempotency and Sagas
- ✓ Security, resilience and observability
- ✓ Pipelines and OpenShift
- ✓ Copilot as a reviewed coding assistant

NOW Let AI agents act on these services — with limits, approvals and audit.



OUR PATH THROUGH MODULE 10

- 1 What agents are and how they loop
- 2 Copilot, Foundry and grounding
- 3 Tools, loops and stop conditions
- 4 Single- and multi-agent designs
- 5 Orchestrating Spring Boot services
- 6 Guardrails, approval and policy
- 7 Reliability, use cases and the big picture

Part 1

Part 2

Part 3

Part 3

Part 4

Part 5

Part 5

KEY TAKEAWAY Module 9 used AI to build services; Module 10 lets AI use them — under control.

PART 1: AGENT FUNDAMENTALS

What turns a model into an agent — and why that changes the risk.



SO FAR

- ✓ Copilot drafts code for people to review
- ✓ Services expose governed APIs
- ✓ Every action is secured and observable

NEXT QUESTION

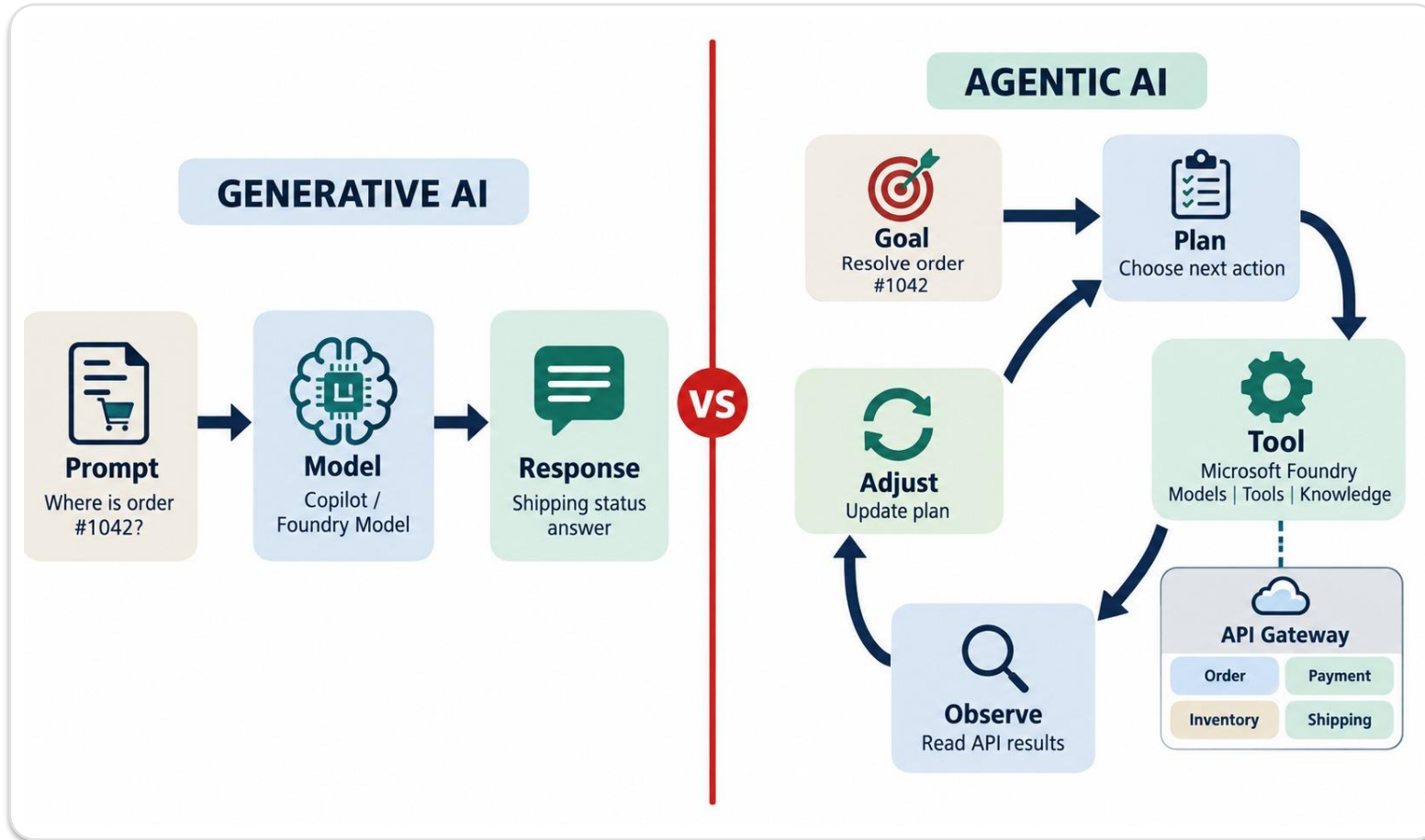
What is an AI agent — and how is it different from a chatbot or Copilot?

IN THIS PART

- 1 Generative vs. agentic AI
- 2 From chatbot to autonomous agent
- 3 Core components of an agent
- 4 The execution loop
- 5 Levels of autonomy
- 6 Exercise: chatbot, copilot or agent?

WHAT IS AGENTIC AI?

The difference between generative AI and goal-driven agents.



Generative AI

One prompt in, one response out — a person acts on it.

Agentic AI

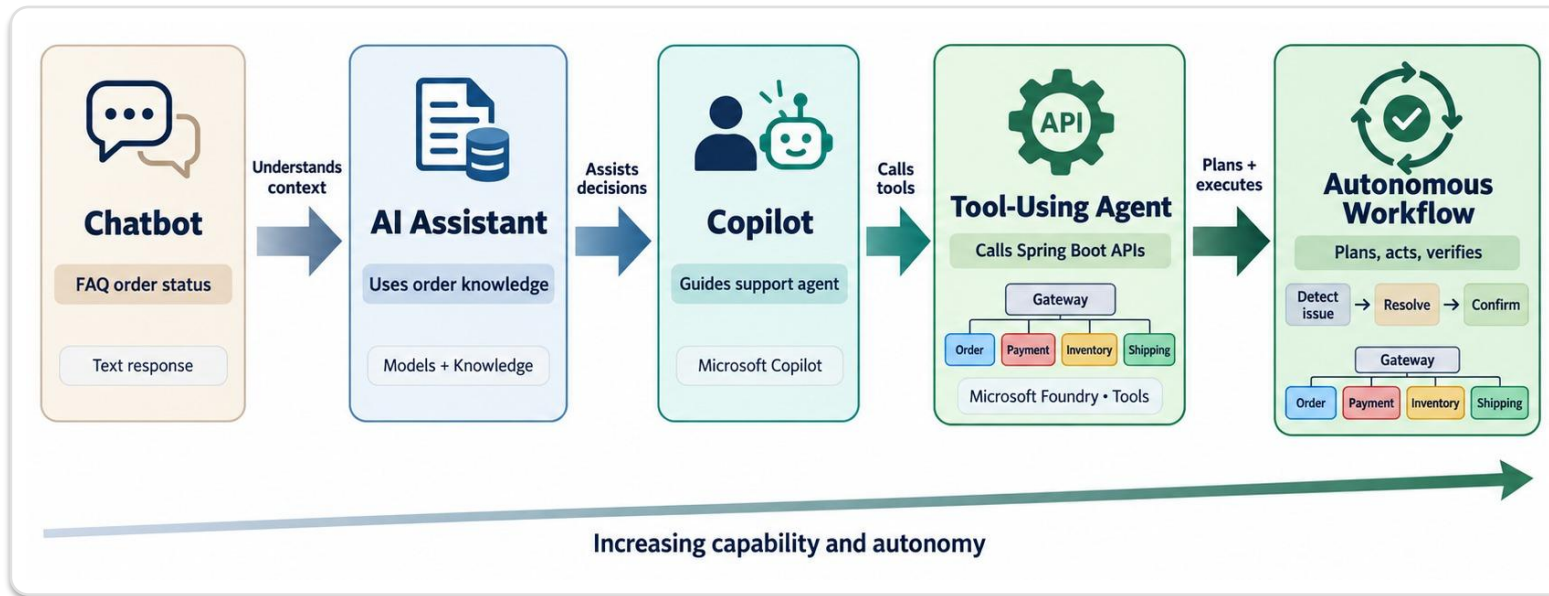
A goal in; the agent plans, calls tools, reads results and adjusts.

DEFINING IDEA The agent takes part in the actions, not just the answer.

KEY TAKEAWAY An agent takes part in a multi-step loop of decisions and actions — not just one response.

FROM PROMPT-RESPONSE AI TO AUTONOMOUS AGENTS

Prompt → response versus goal → plan → act → observe → iterate.



Chatbot and assistant

Answer questions; people take every action.

Copilot

Suggests and drafts inside a person's workflow.

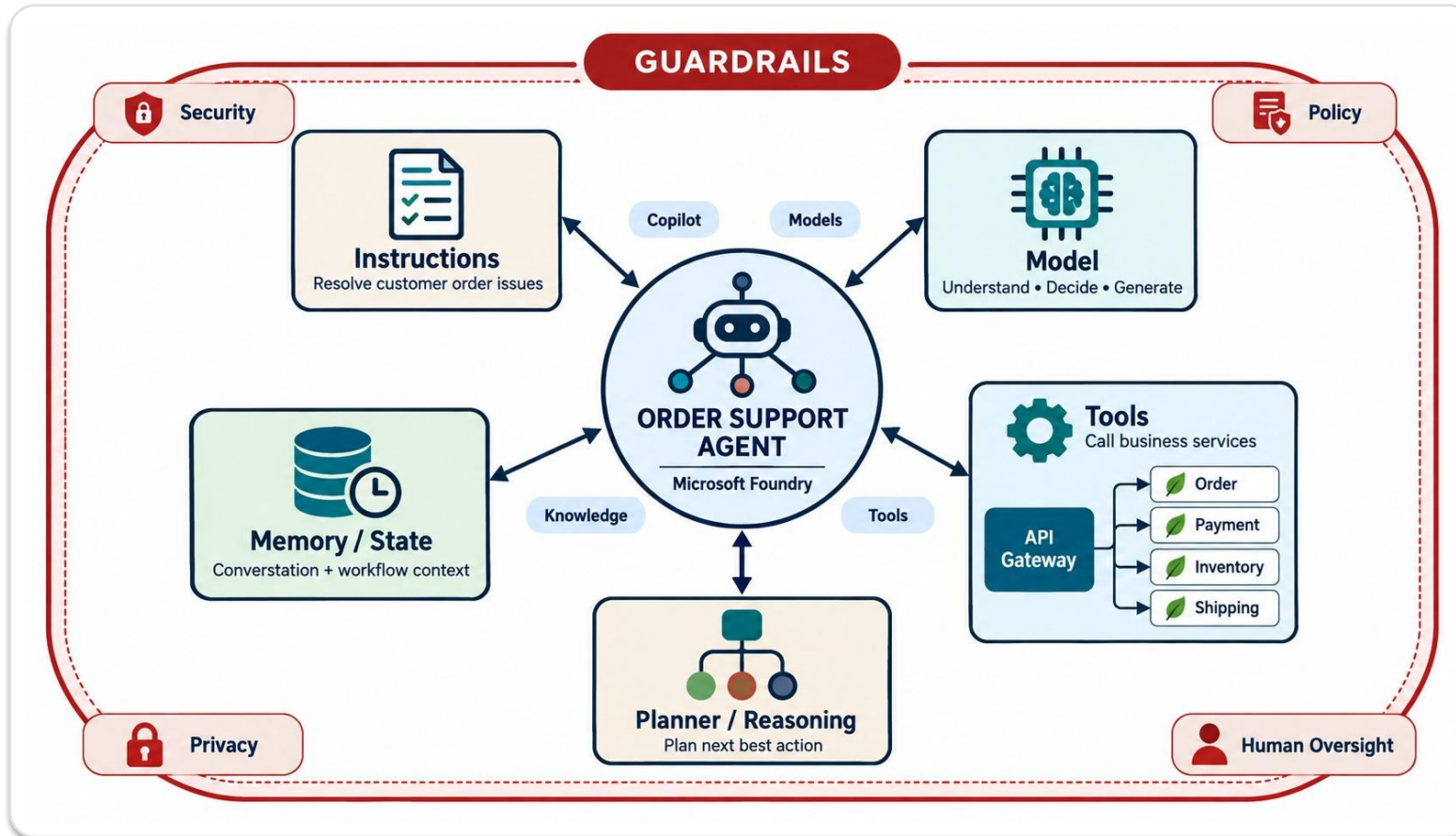
Agent and autonomous workflow

Calls tools and completes steps — with limits.

KEY TAKEAWAY AI moves from answering prompts to pursuing goals — one step of autonomy at a time.

CORE COMPONENTS OF AN AI AGENT

Model, instructions, memory/state, tools, planner and guardrails.



Instructions

Role, goal, allowed actions and when to stop or escalate.

Tools

Narrow, approved functions — not raw database access.

State

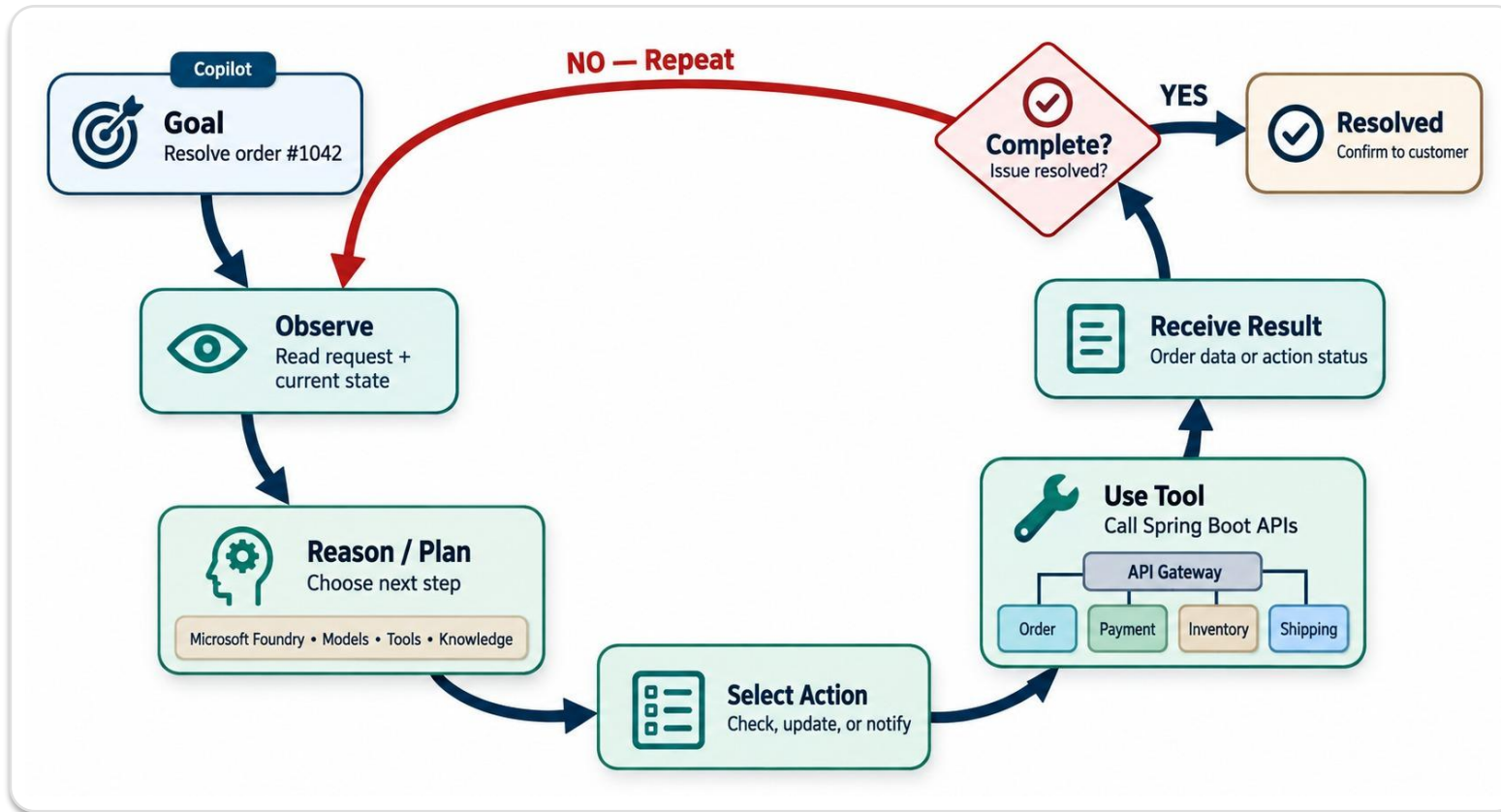
What the agent has done so far in this task.

GUARDRAILS wrap everything: identity, policy, approval, audit.

KEY TAKEAWAY An agent is a model plus instructions, state, tools and guardrails.

AGENTIC AI EXECUTION LOOP

Perceive → reason/plan → act → observe → adjust.



A WORKED EXAMPLE

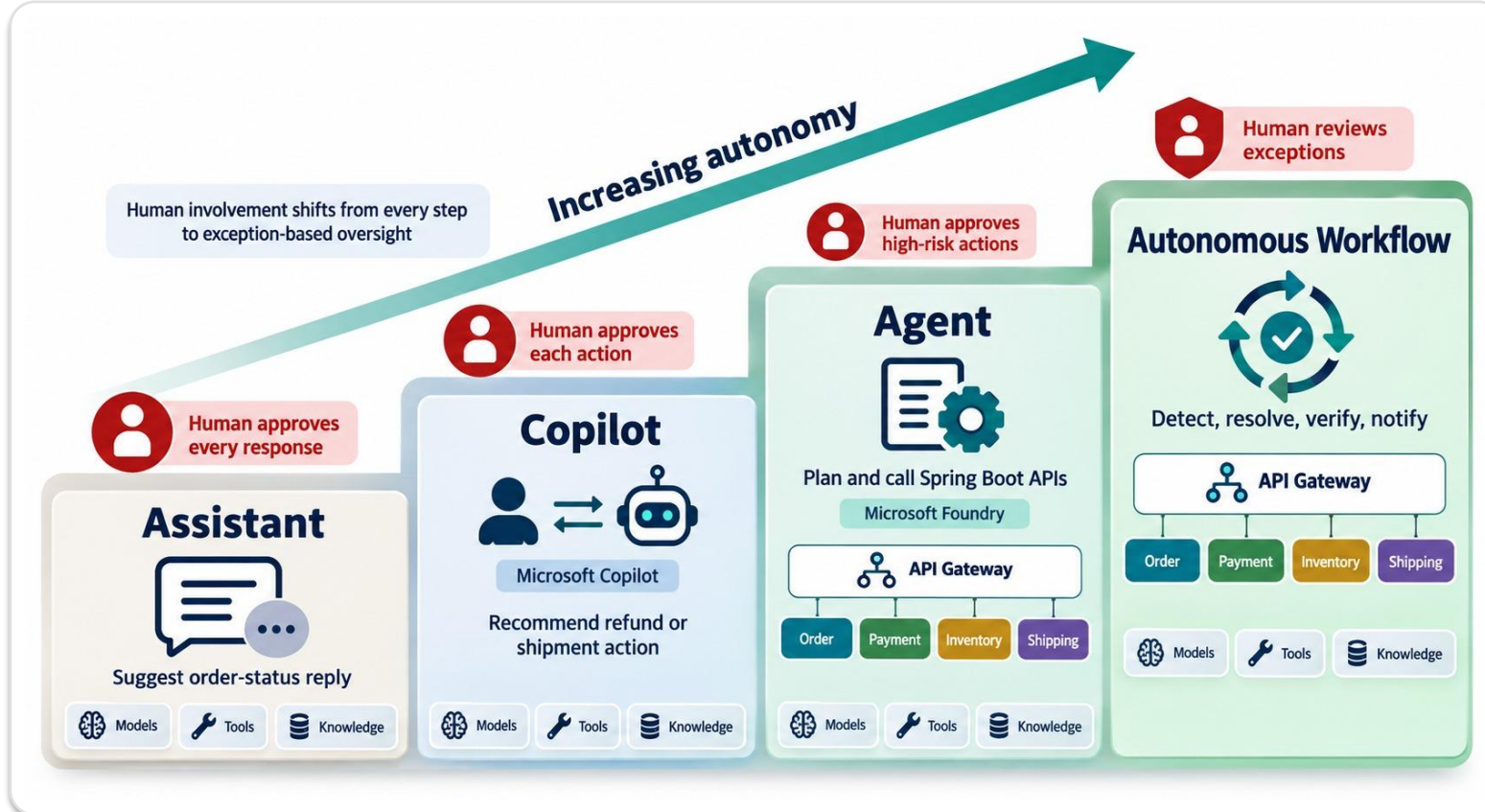
- ✓ Goal: "Where is order 1045?"
- ✓ Plan: look up the order
- ✓ Act: call getOrder
- ✓ Observe: status SHIPPED
- ✓ Act: call getShipment
- ✓ Done: answer with tracking

STOP when done, blocked, or out of steps.

KEY TAKEAWAY Agents loop — observe, reason, act, read the result — until the goal is complete.

LEVELS OF AI AUTONOMY

Assistant, copilot, semi-autonomous agent and autonomous workflow.



Low risk

Read-only lookups can run without approval.

Medium risk

Drafts and recommendations — a person confirms.

High risk

Refunds, freezes, money movement — always approved.

RULE Match autonomy to risk, not to capability.

KEY TAKEAWAY Match autonomy to risk: the riskier the action, the stronger the control.

EXERCISE: CHATBOT, COPILOT OR AGENT?

Classify each system — and decide the control it needs.

PRACTICE EXERCISE

10 MIN

SCENARIO

Five proposals from the business:

- 1 FAQ bot answering fee questions
- 2 Drafts replies for support staff
- 3 Looks up order + shipment, answers
- 4 Cancels orders when customers ask
- 5 Nightly: finds stuck payments, retries

YOUR TASKS

- 1 Label each: chatbot, copilot, agent or autonomous workflow
- 2 Mark which actions change state
- 3 Choose the control each needs

EXPECTED OUTCOME

- ✓ 1 chatbot; 2 copilot
- ✓ 3 agent — read-only tools
- ✓ 4 agent — needs approval
- ✓ 5 autonomous — limits and alerts
- ✓ State changes need identity and audit

KEY TAKEAWAY The label matters less than the actions: anything that changes state needs authority, limits and audit.

PART 2: THE MICROSOFT AGENT ECOSYSTEM

Where agents are built, which models they use and how they are grounded.



SO FAR

- ✓ Agents loop toward a goal with tools
- ✓ Instructions, state and guardrails shape behavior
- ✓ Autonomy must match risk

NEXT QUESTION

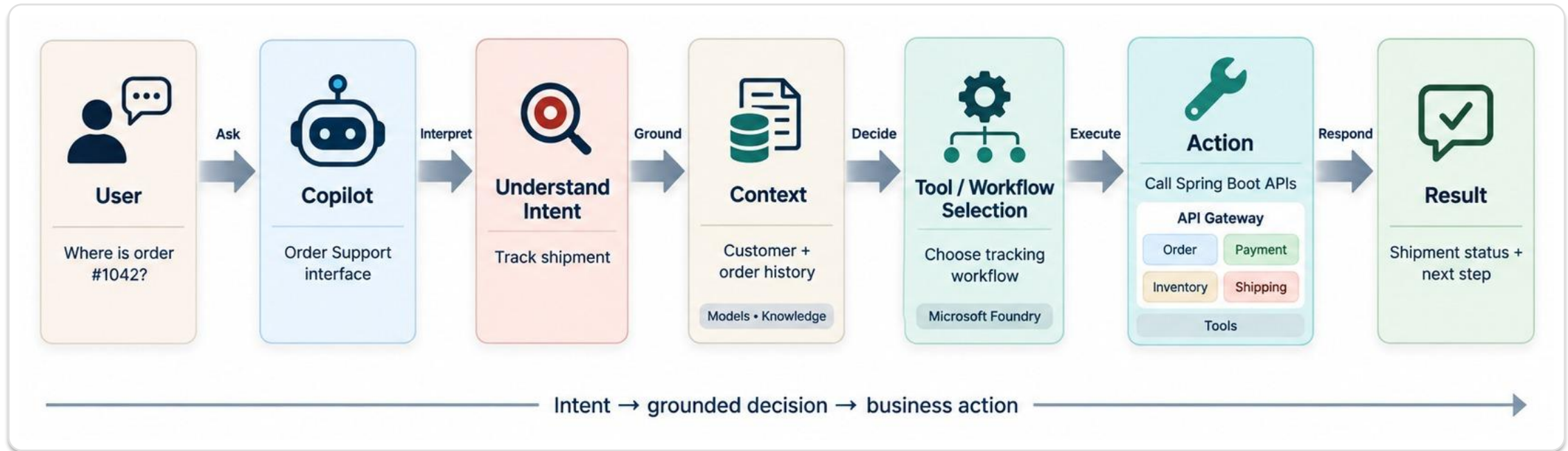
Which Microsoft services do we use to build agents — and what does each do?

IN THIS PART

- 1 Copilot as an agent
- 2 An order-support agent workflow
- 3 The Microsoft AI services landscape
- 4 Microsoft Foundry and models
- 5 Tool calling with Azure OpenAI
- 6 Grounding with enterprise data

USING COPILOT AS AN AI AGENT

Copilot as an interface that reasons over context and invokes tools and workflows.



Knowledge

Documents and sites the agent can read — it tells the agent something.

Tools and actions

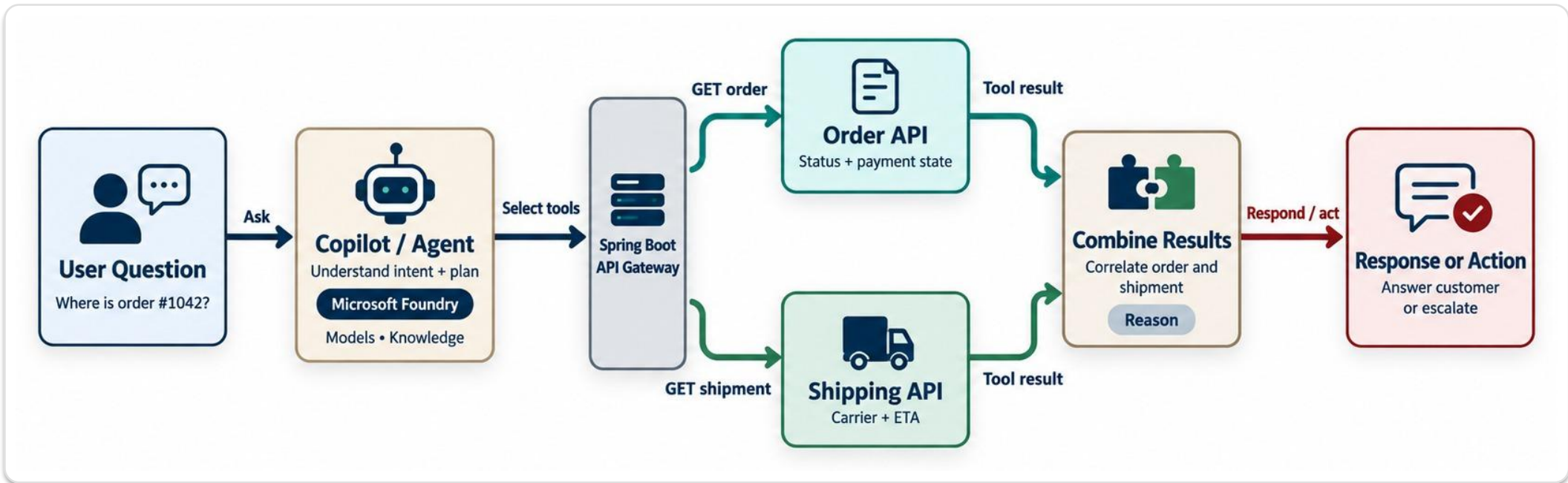
Connectors and APIs — they let the agent do something.

COPILOT STUDIO builds these agents with low code.

KEY TAKEAWAY Knowledge tells the agent something; tools let it do something.

COPILOT AGENT WORKFLOW

User intent → context → reasoning → tool call → result → response/action.



Intent

"Where's my order, and can I change the address?"

Calls

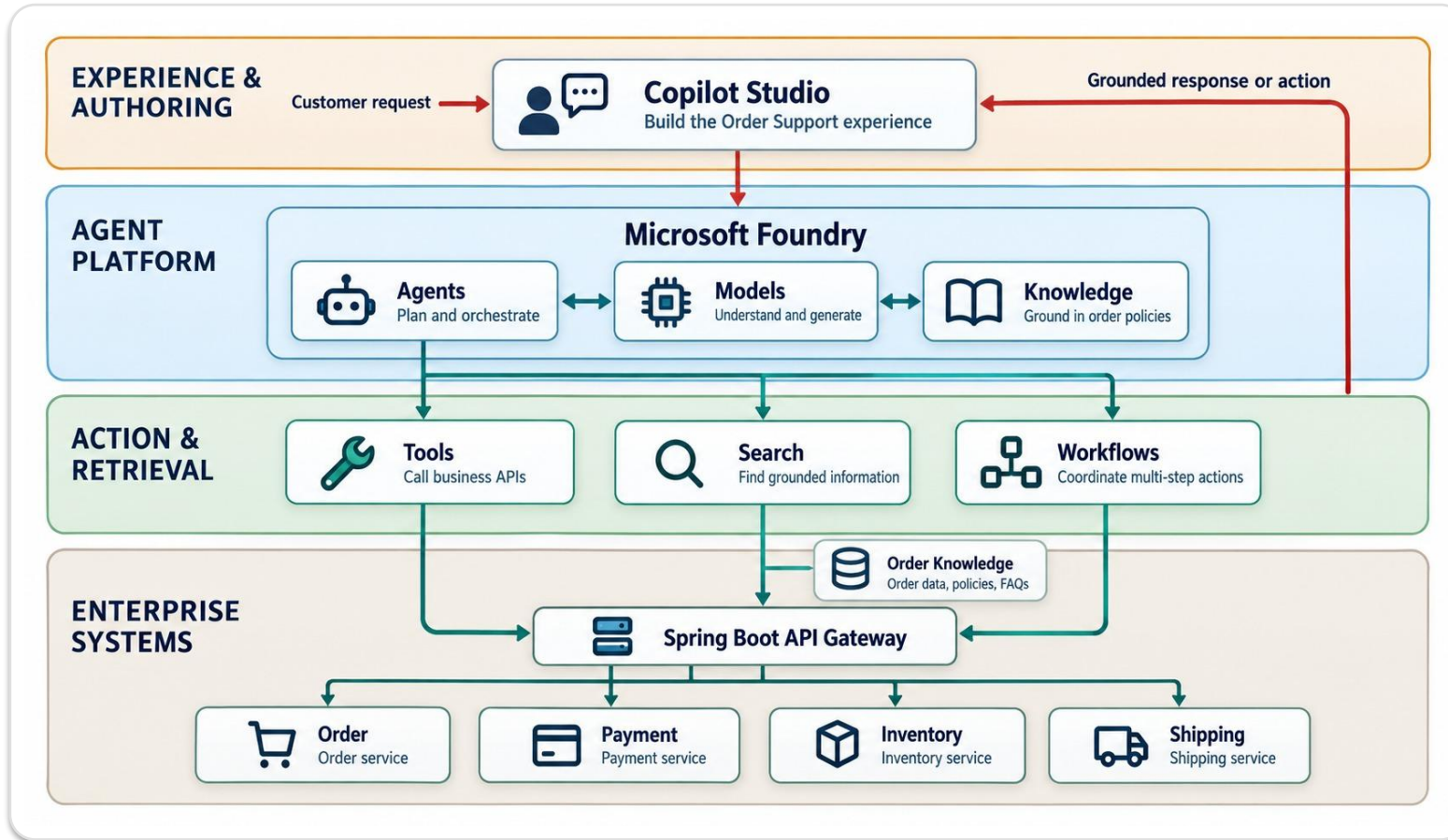
getOrder, getShipment — then, if allowed, updateAddress.

STILL GOVERNED Tools run with the user's permissions, not admin rights.

KEY TAKEAWAY The agent turns natural-language intent into calls to governed enterprise APIs.

MICROSOFT AI SERVICES LANDSCAPE

Copilot Studio, Foundry, models, tools, knowledge and enterprise systems.



Copilot Studio

Build and publish agents with low code and connectors.

Microsoft Foundry

Pro-code agents, model deployments and evaluation.

Azure AI Search

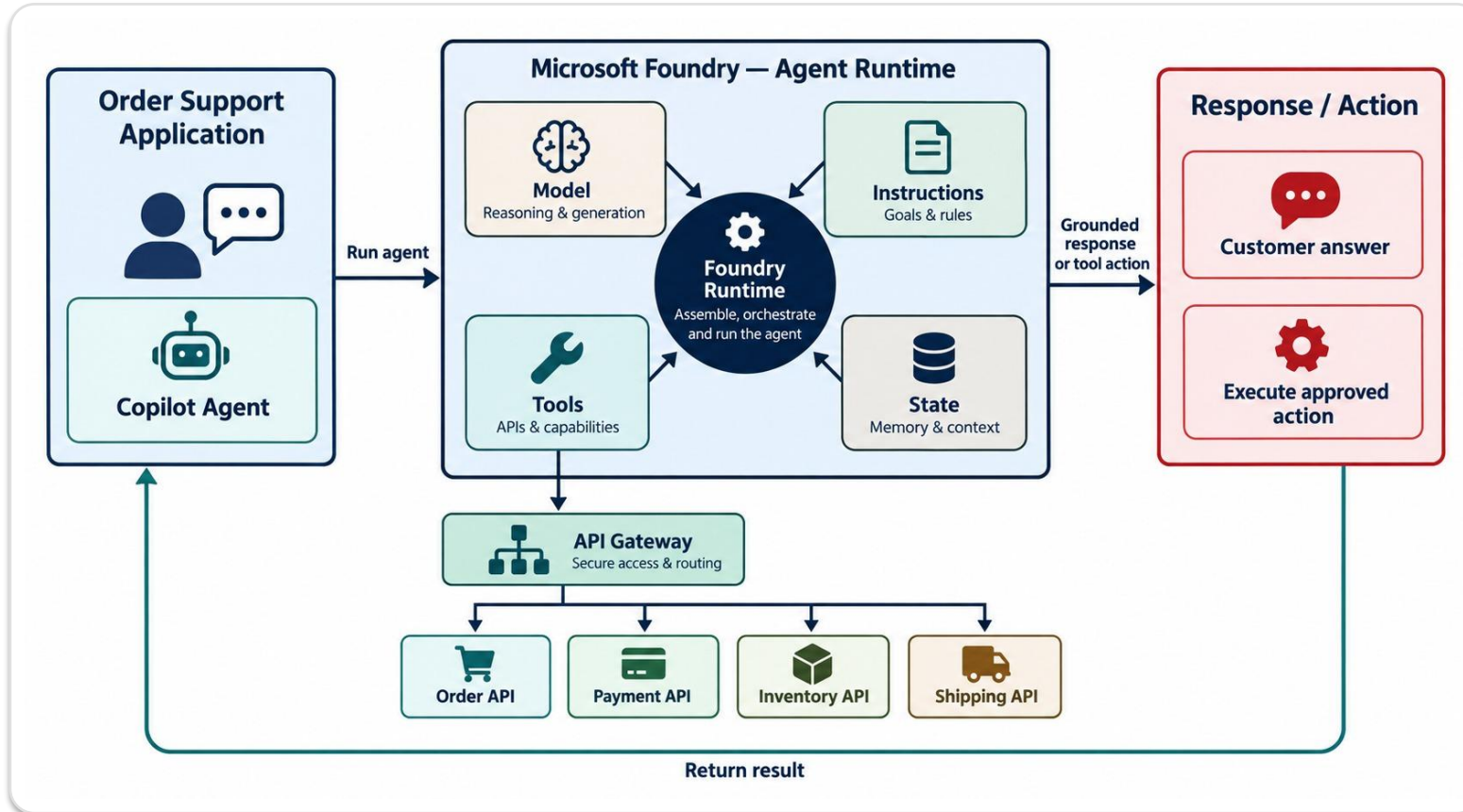
Retrieval over enterprise content for grounding.

YOUR SYSTEMS stay the source of truth.

KEY TAKEAWAY Microsoft's AI services give agents models, tools and knowledge on top of your systems.

MICROSOFT FOUNDRY AND MODEL INTEGRATION

Model access, deployment, prompts, grounding and evaluation.



WHAT FOUNDRY PROVIDES

Model catalog

Deployments

Agent Service

Evaluations

Content safety

Tracing

Model choice

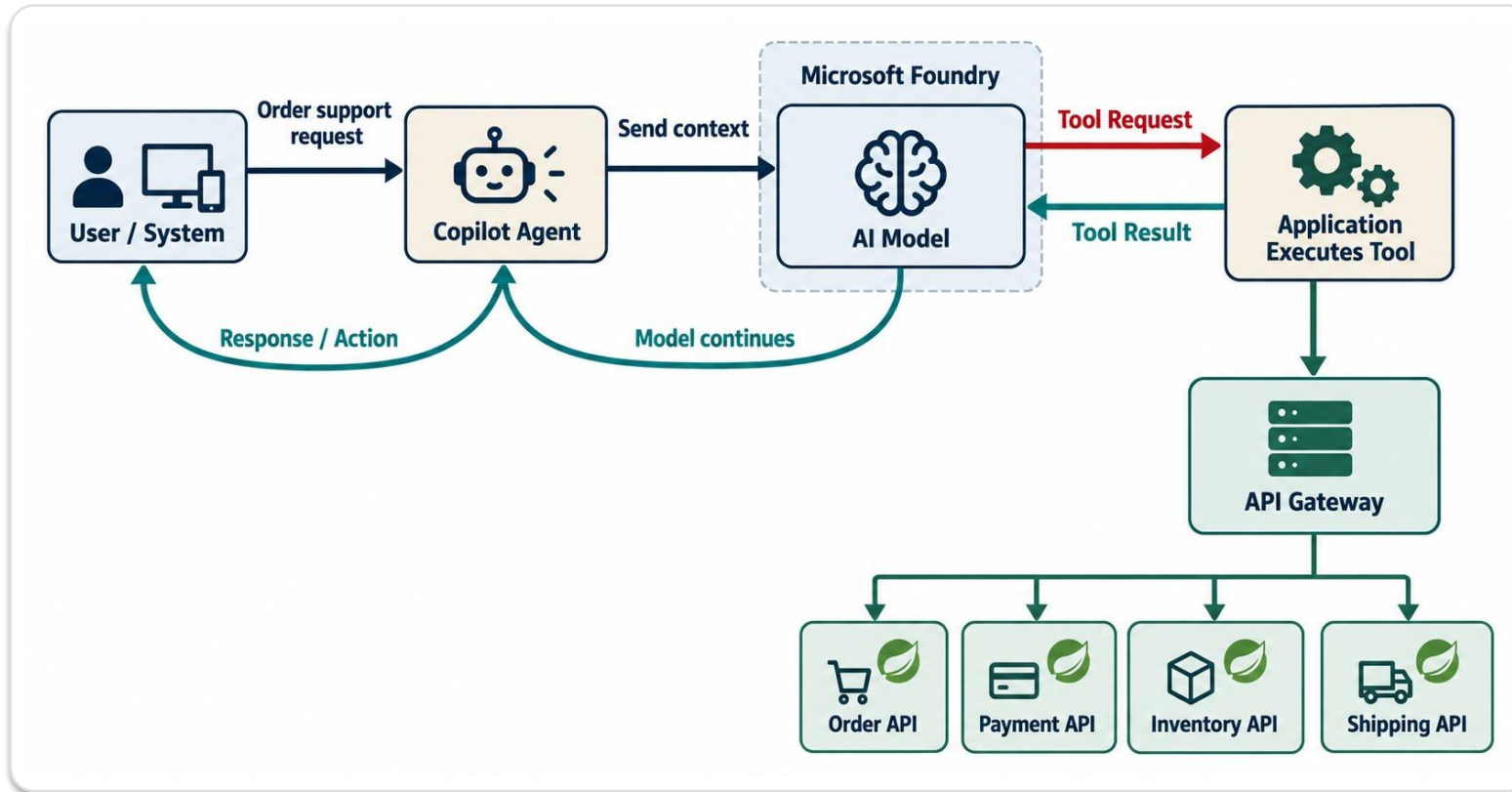
Pick for quality, latency and cost — then evaluate.

MORE THAN AN LLM CALL Instructions, tools, state and evaluation.

KEY TAKEAWAY Enterprise AI needs model choice, instructions, tools, state and evaluation — not just an LLM call.

AZURE OPENAI IN AN AGENTIC ARCHITECTURE

The LLM as the reasoning engine inside an enterprise agent.



HOW TOOL CALLING REALLY WORKS

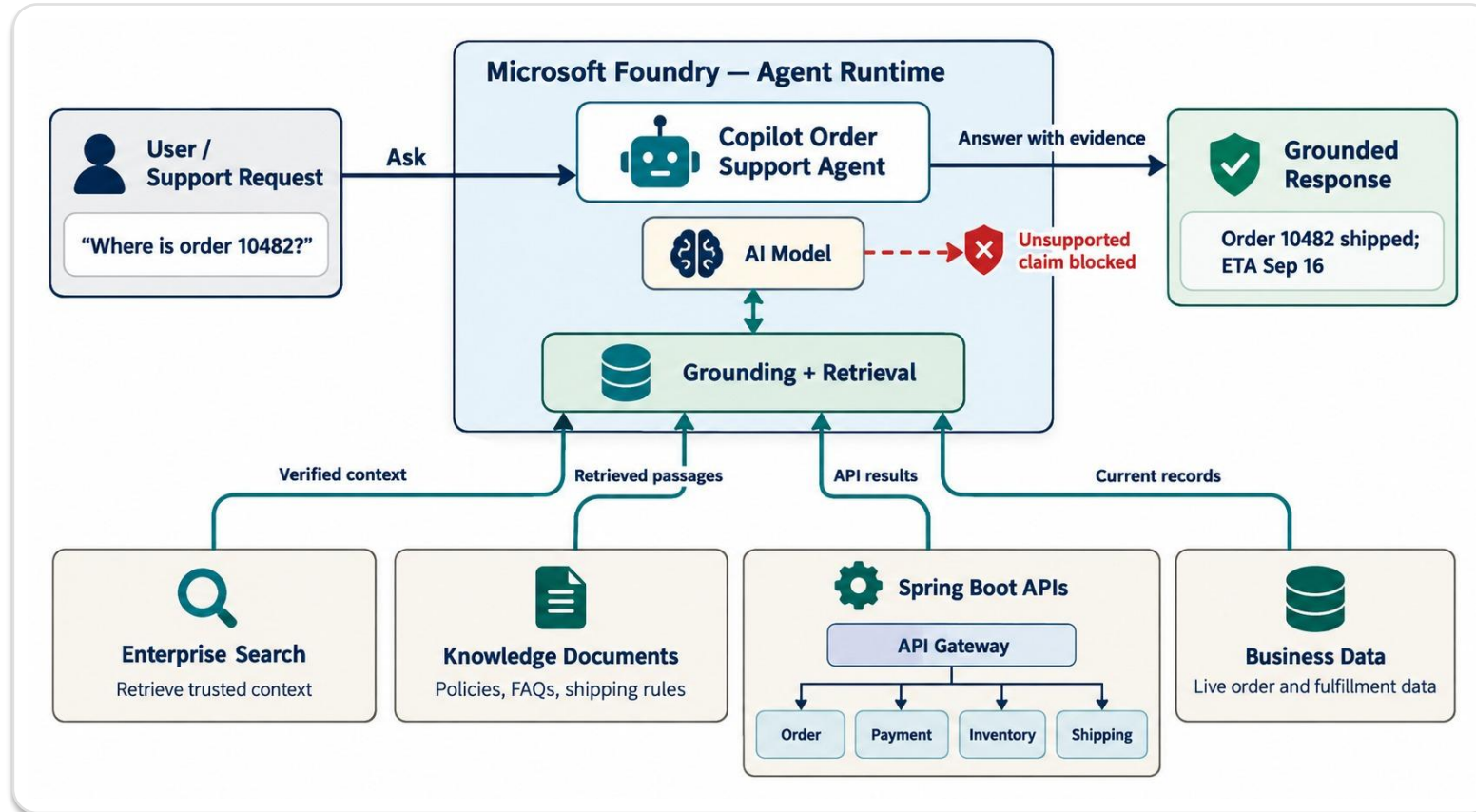
- ✓ The model never calls your API directly
- ✓ It returns a tool request with arguments
- ✓ Your code validates and executes it
- ✓ The result goes back to the model

CONTROL stays in the application.

KEY TAKEAWAY The model reasons and requests tools; the application executes them under control.

GROUNDING AGENTS WITH ENTERPRISE DATA

Search/RAG, internal knowledge, APIs and databases.



Knowledge grounding

Policies, runbooks and FAQs retrieved through search.

Live grounding

Current order and payment status from service APIs.

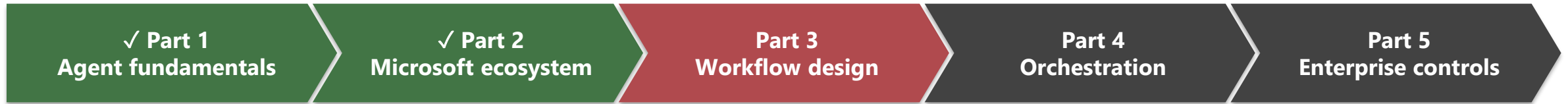
GROUNDING PREVENTS

- ✗ Guessing an order status
- ✗ Quoting an old policy

KEY TAKEAWAY Ground agents in enterprise data — current facts come from services, not model memory.

PART 3: DESIGNING AGENT WORKFLOWS

Give an agent tools, a loop and clear limits.



SO FAR

- ✓ Copilot Studio and Foundry build agents
- ✓ The model requests; the app executes
- ✓ Grounding supplies current facts

NEXT QUESTION

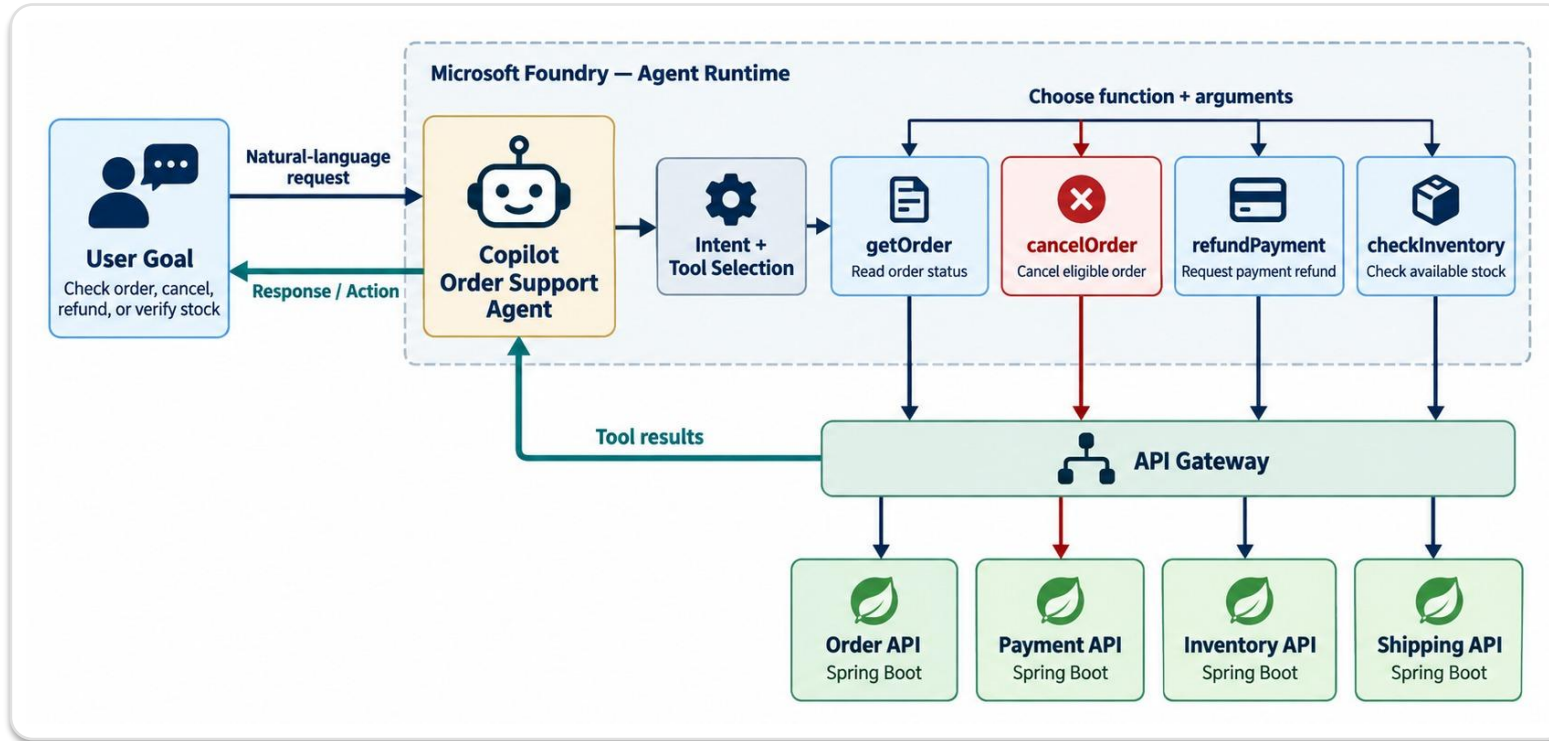
How do we design what an agent may do — and when it must stop?

IN THIS PART

- 1 Tools and function calling — a tool definition
- 2 Autonomous workflow design — a bounded loop
- 3 Single-agent and multi-agent patterns
- 4 State, memory and context
- 5 Exercise: design tools and stop conditions

TOOLS AND FUNCTION CALLING

How agents invoke APIs and business operations.



Name and description

Tell the model when — and when not — to use it.

Schema

Strict, typed inputs the application can validate.

Scope

Read tools freely; state-changing tools need approval.

CRITICAL Approved capabilities only — never raw infrastructure.

KEY TAKEAWAY Models choose from approved capabilities — never unlimited access to infrastructure.

DEFINING TOOLS FOR AN ORDER-SUPPORT AGENT

Narrow, typed, well-described tools — one to read, one to request an action.

✓ **get_order_status (read-only)**

```
{
  "type": "function",
  "name": "get_order_status",
  "description": "Return status and shipment of ONE order.
    Read-only. Use before answering status questions.",
  "parameters": {
    "type": "object",
    "properties": {
      "orderId": { "type": "string",
        "pattern": "^ORD-[0-9]{4,}$" }
    },
    "required": ["orderId"],
    "additionalProperties": false
  }
}
```

Descriptions

The model picks tools by reading them — make them precise.

request_order_cancellation (action)

```
{
  "type": "function",
  "name": "request_order_cancellation",
  "description": "Create a cancellation REQUEST that a
    person must approve. Never cancels directly.",
  "parameters": {
    "type": "object",
    "properties": {
      "orderId": { "type": "string" },
      "reason": { "type": "string", "maxLength": 200 }
    },
    "required": ["orderId", "reason"],
    "additionalProperties": false
  }
}
```

Strict schemas

Patterns and no extra properties block junk input.

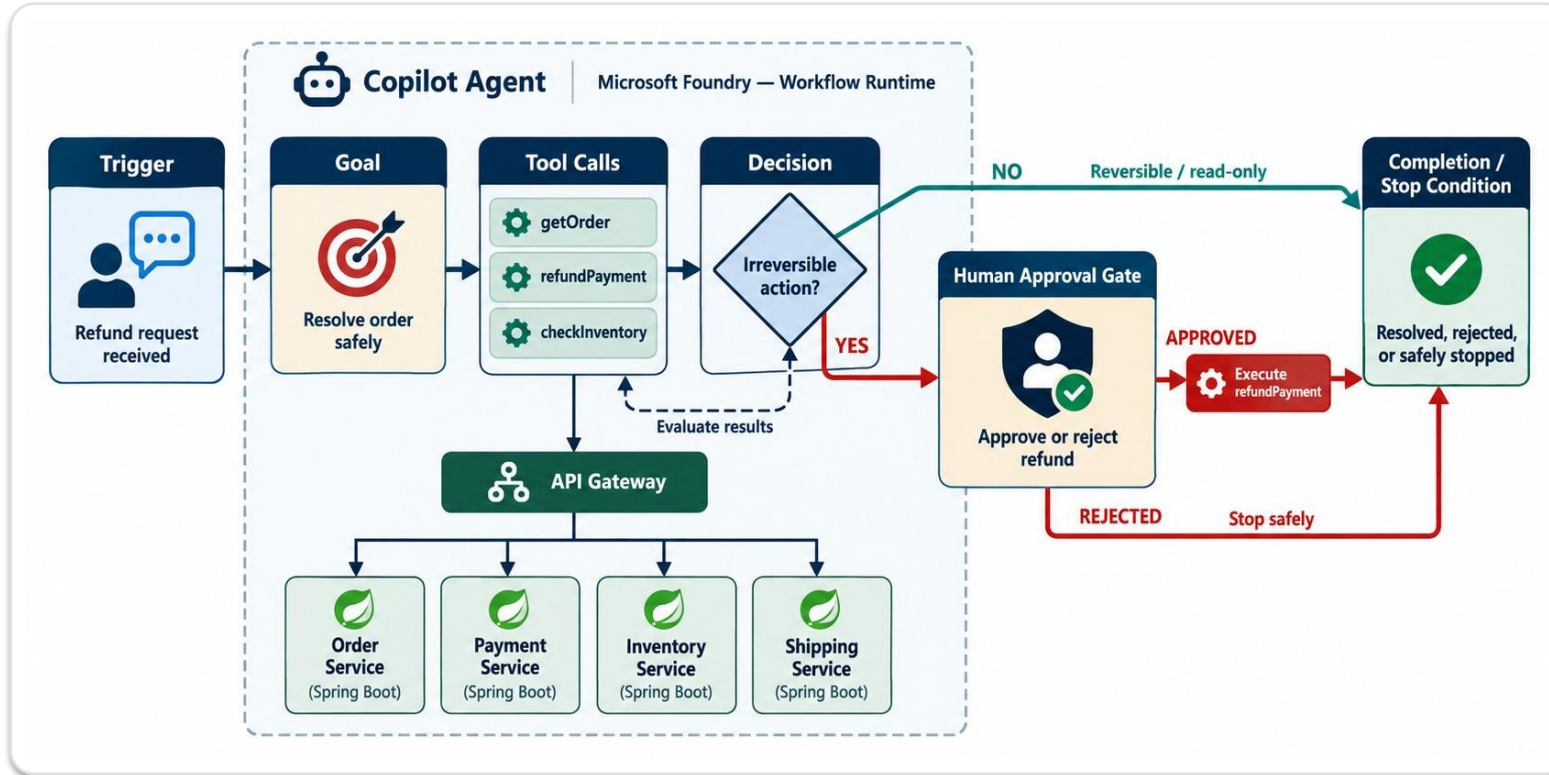
Requests, not actions

The risky tool only creates a pending request.

KEY TAKEAWAY Design tools like public APIs: narrow purpose, strict schema, and no direct high-risk actions.

DESIGNING AUTONOMOUS WORKFLOWS

Goal, steps, tools, conditions, state, retry and termination.



BEFORE YOU BUILD

- ✓ Explicit goal and trigger
- ✓ Allowed tools only
- ✓ Decision authority defined
- ✓ Approval gate for risky steps
- ✓ Step, time and cost limits

STOP CONDITIONS Done, blocked, over budget — or escalate.

KEY TAKEAWAY Autonomous workflows need an explicit goal, trigger, tools, authority, approvals and stop condition.

A BOUNDED AGENT LOOP

Allowed tools, approval for state changes, and hard limits on steps and time.

✓ AgentRunner.java (simplified)

```
public AgentOutcome run(Goal goal, User user) {
    AgentRun run = AgentRun.start(goal, MAX_STEPS, Duration.ofSeconds(60));

    while (true) {
        ModelTurn turn = model.next(run.context(), tools.allowedFor(user));
        if (turn.isFinalAnswer())
            return run.complete(turn.answer());

        ToolCall call = turn.toolCall();
        if (!tools.isAllowed(call, user))
            return run.escalate("TOOL_NOT_ALLOWED");
        if (call.changesState())
            return run.awaitApproval(call);           // a person decides

        run.record(call, tools.execute(call));       // timeout + audit inside

        if (run.steps() >= MAX_STEPS || run.expired())
            return run.escalate("LIMIT_REACHED");
    }
}
```

Allow-list

Tools are filtered by the user's permissions first.

Approval

Any state change pauses for a human decision.

Limits

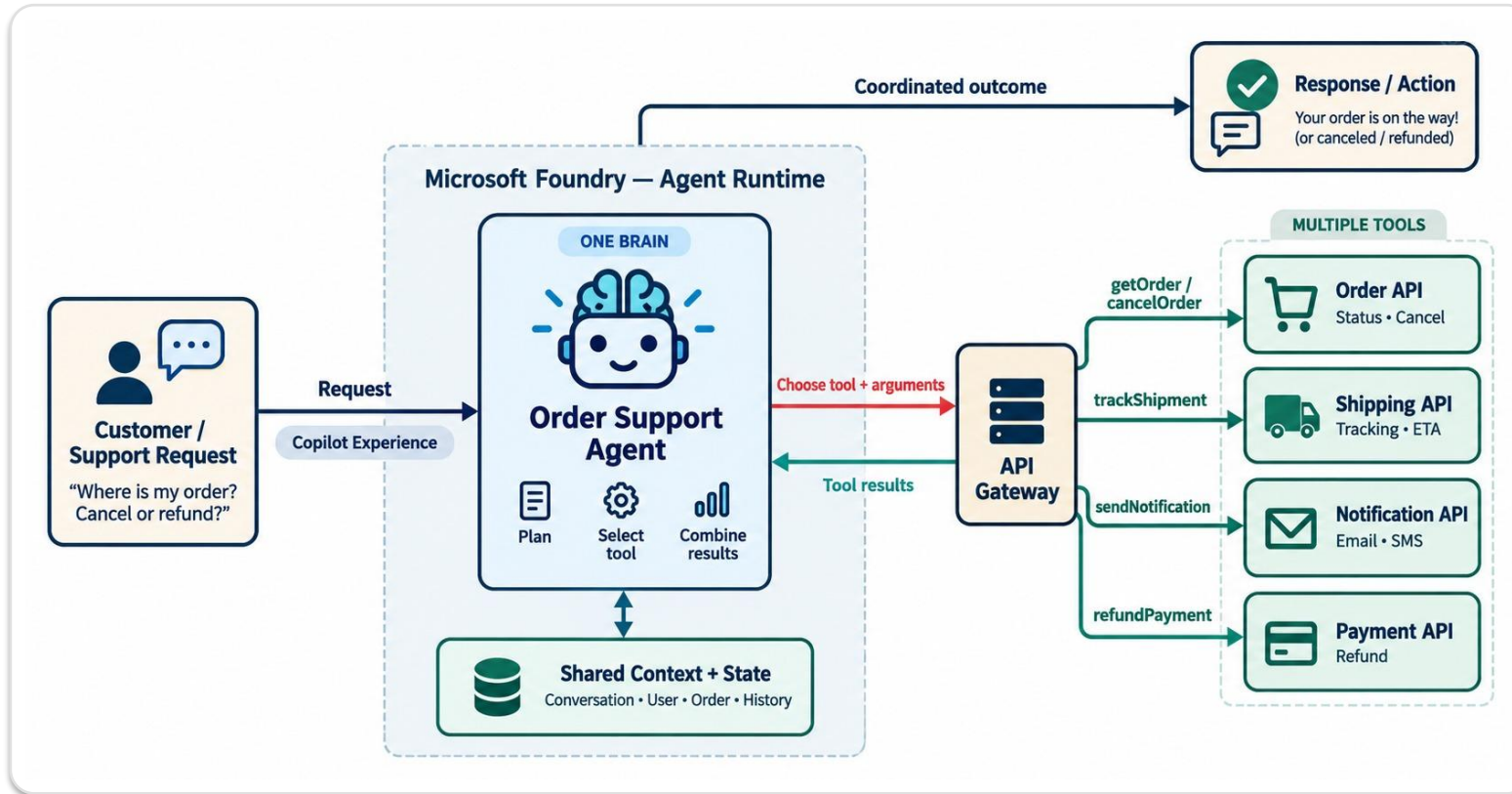
Step and time limits stop infinite loops.

AUDIT Every call and result is recorded.

KEY TAKEAWAY An agent loop is only safe with an allow-list, an approval gate and hard stop conditions.

SINGLE-AGENT WORKFLOW DESIGN

One agent coordinating several business tools.



ADVANTAGES

- ✓ One set of instructions
- ✓ One audit trail
- ✓ Easier to test and debug

Keep tools focused

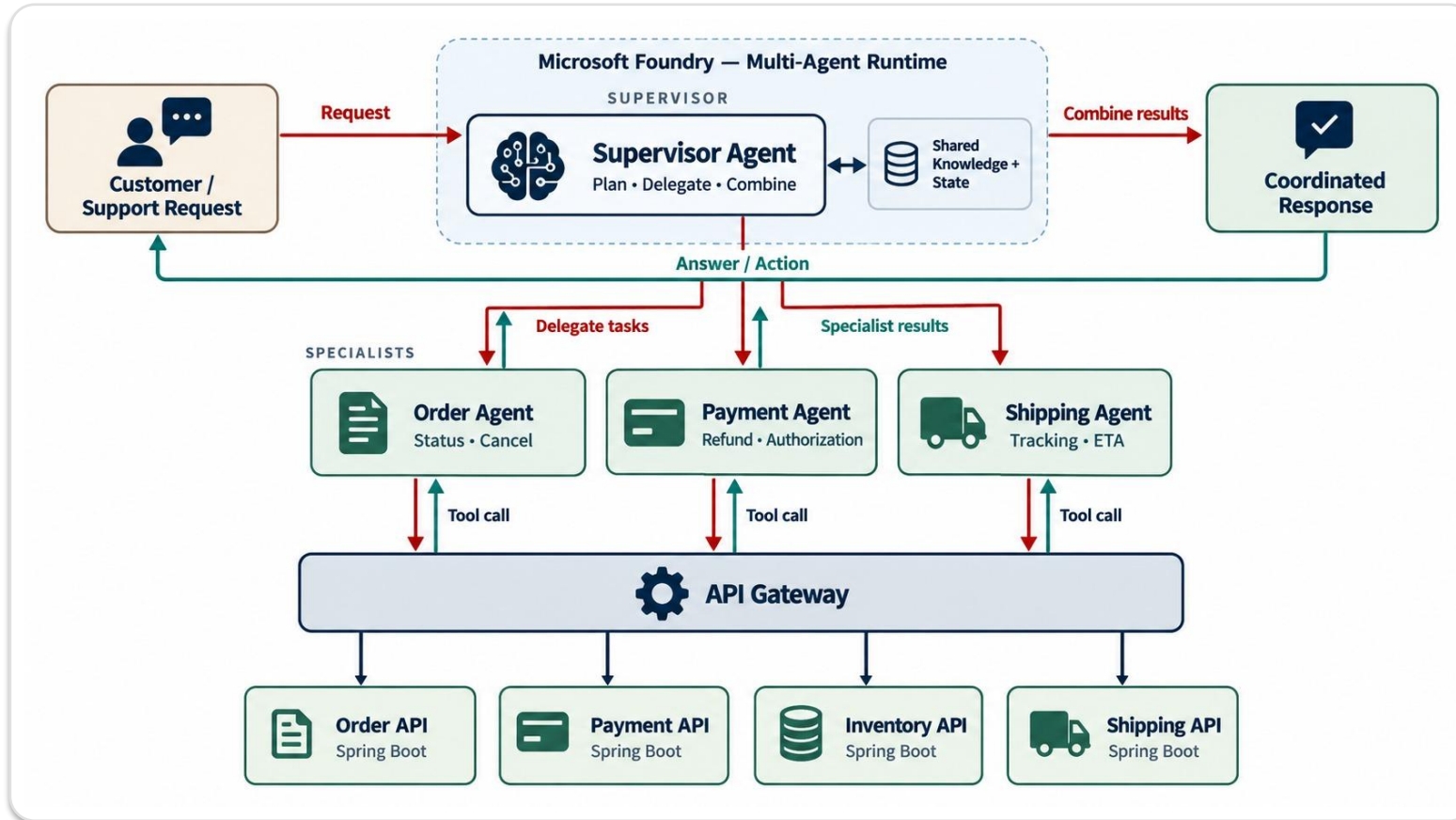
Four to ten well-described tools beats forty.

START HERE Split into more agents only when you must.

KEY TAKEAWAY One agent owning the goal is the simplest place to start.

MULTI-AGENT WORKFLOW DESIGN

Specialized agents coordinating tasks.



Supervisor

Breaks the goal down and routes work to specialists.

Specialists

Order, Payment and Shipping agents with narrow tools.

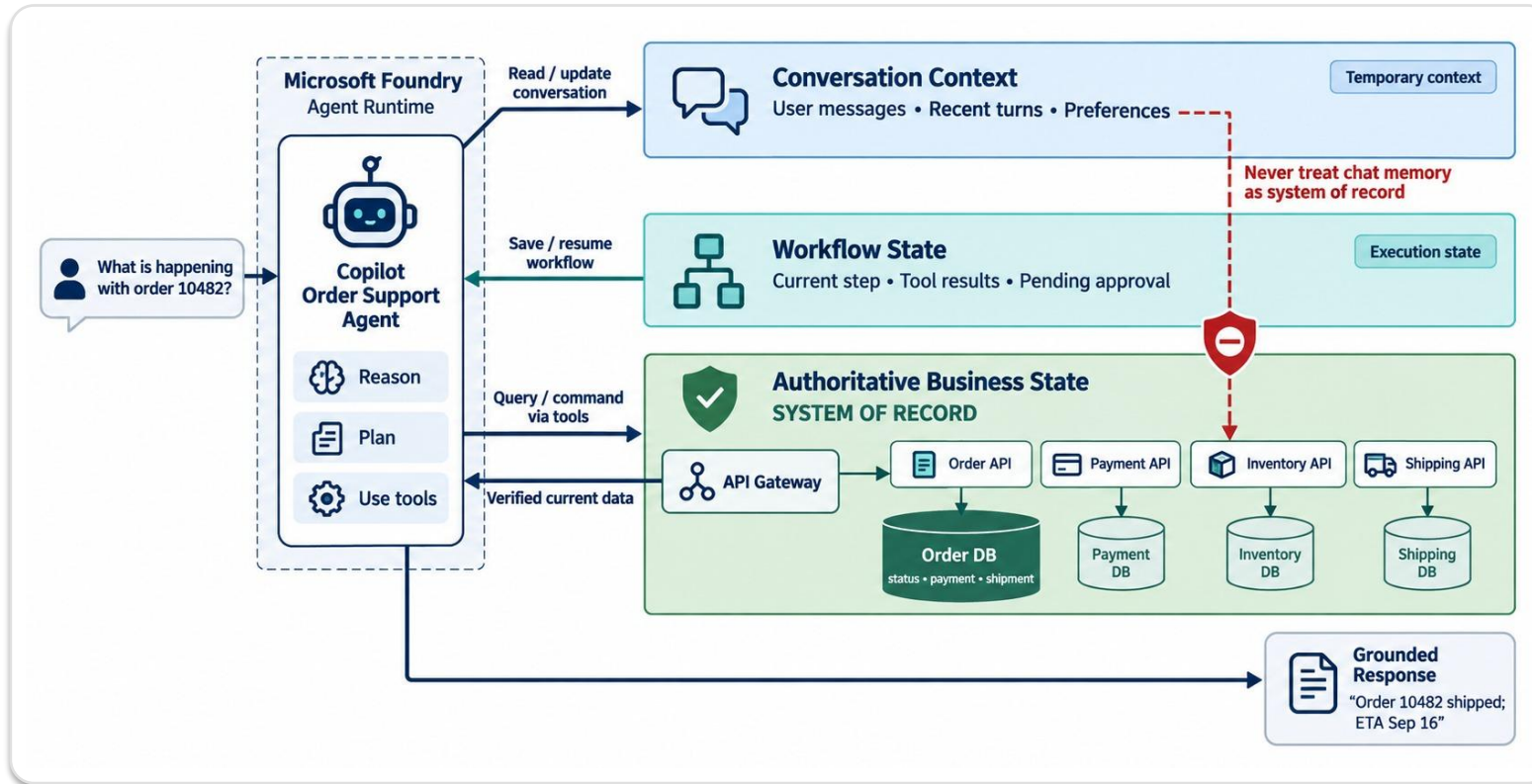
TRADE-OFFS

- ! More model calls and cost
- ! Harder tracing
- ! Agents misunderstanding each other

KEY TAKEAWAY Specialized agents can help complex domains — but every extra agent adds cost and complexity.

AGENT STATE, MEMORY, AND CONTEXT

Conversation state, workflow state, durable state and context limits.



Conversation context

What the user said — trimmed to fit the context window.

Workflow state

Steps done, pending approvals — stored durably.

Business state

Orders and payments — only in the owning services.

KEY TAKEAWAY Keep conversation, workflow and business state separate — services own the facts.

EXERCISE: DESIGN TOOLS AND STOP CONDITIONS

Turn a business goal into a bounded agent design.

PRACTICE EXERCISE

12 MIN

SCENARIO

Support wants an agent that handles “my order hasn't arrived” requests.

It may look up orders and shipments, reship low-value orders and request refunds.

YOUR TASKS

- 1 List the tools — read or state-changing
- 2 Decide which actions need human approval
- 3 Write three stop conditions
- 4 Say where workflow state is stored

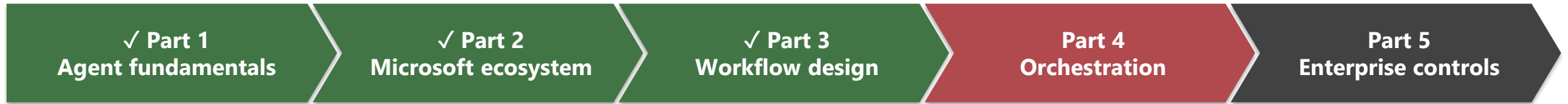
EXPECTED OUTCOME

- ✓ Read: get_order, get_shipment
- ✓ Action: request_reship, request_refund
- ✓ Approval: refunds and reships above a limit
- ✓ Stop: resolved, 8 steps, unclear intent → human
- ✓ State in a workflow store — not the chat

KEY TAKEAWAY Define read and action tools, approval rules and stop conditions before any prompt is written.

PART 4: MICROSERVICE ORCHESTRATION

Agents act through our services — never around them.



SO FAR

- ✓ Tools are narrow, typed and described
- ✓ Loops have allow-lists, approvals and limits
- ✓ Services own the business state

NEXT QUESTION

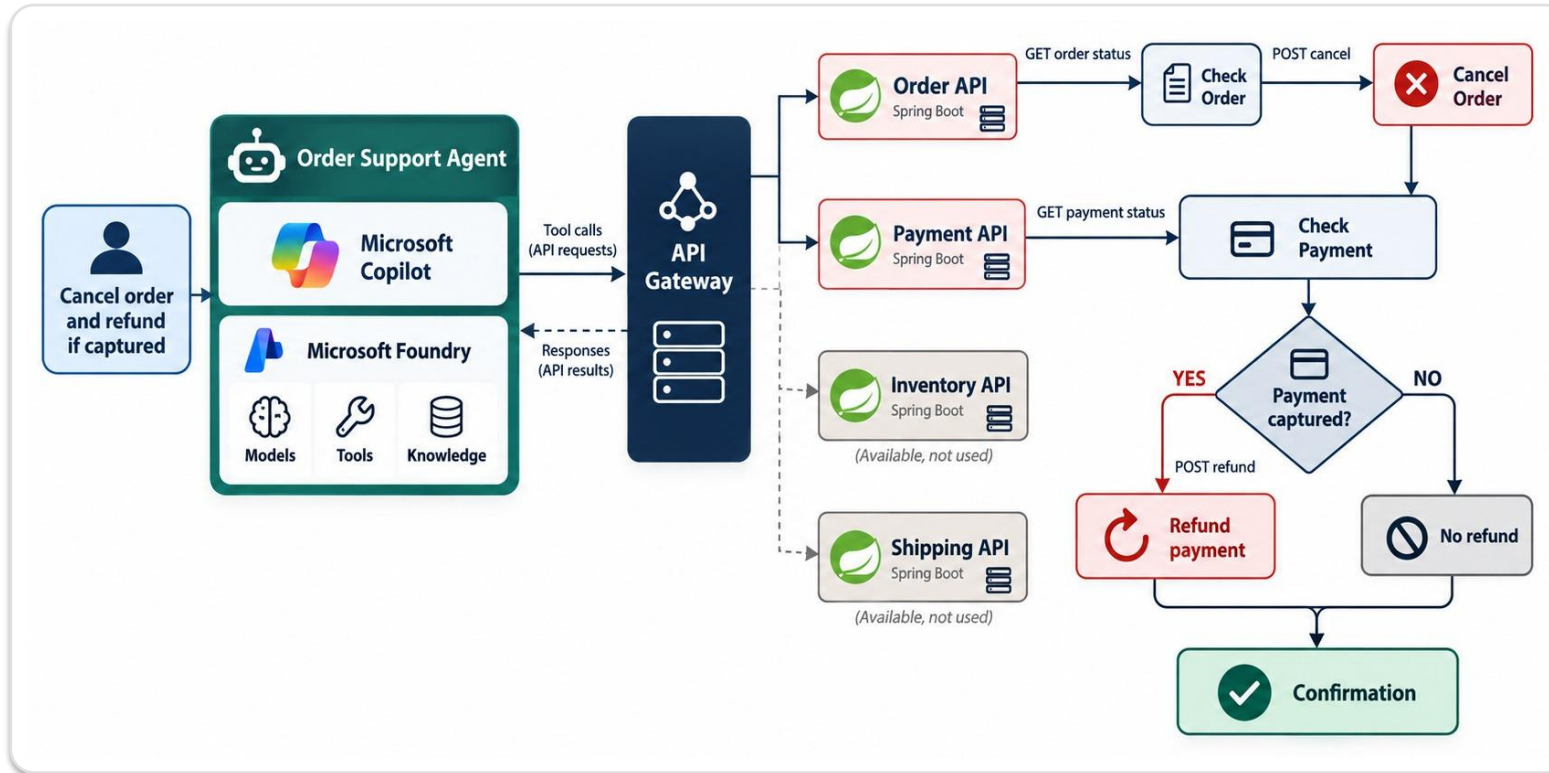
How should an agent use the Order, Payment, Inventory and Shipping services?

IN THIS PART

- 1 AI-assisted API orchestration
- 2 A Spring Boot tool adapter — in code
- 3 Agents with Spring Boot microservices
- 4 Event-driven agentic workflows
- 5 Exercise: investigate a failed payment

AI-ASSISTED API ORCHESTRATION

An agent choosing and calling multiple APIs based on intent.



Decision path

Check order → cancel → check payment → refund only if captured.

Business rules

The Order and Payment services still enforce them — the agent can't skip them.

IDEMPOTENCY Retries must never refund twice.

KEY TAKEAWAY The agent maps business intent to a sequence of API decisions — within the business rules.

A SAFE TOOL ENDPOINT IN SPRING BOOT

The agent requests; the service validates, authorizes, records and waits for approval.

✓ AgentToolController.java

```
@RestController
@RequestMapping("/api/v1/agent-tools/orders")
class AgentToolController {

    @PostMapping("/{orderId}/cancellation-requests")
    @PreAuthorize("hasAuthority('SCOPE_orders.cancel.request')")
    ResponseEntity<CancellationRequest> requestCancellation(
        @PathVariable String orderId,
        @RequestHeader("Idempotency-Key") String key,
        @Valid @RequestBody CancellationReason body,
        @AuthenticationPrincipal Jwt caller) {

        CancellationRequest req = cancellations.request(
            orderId, body.reason(), key, caller.getSubject());
        audit.record("CANCEL_REQUESTED", orderId, caller.getSubject());
        return ResponseEntity.accepted().body(req);    // PENDING_APPROVAL
    }
}
```

Narrow scope

orders.cancel.request — not orders.admin.

Idempotent

A repeated tool call returns the same request.

202 Accepted

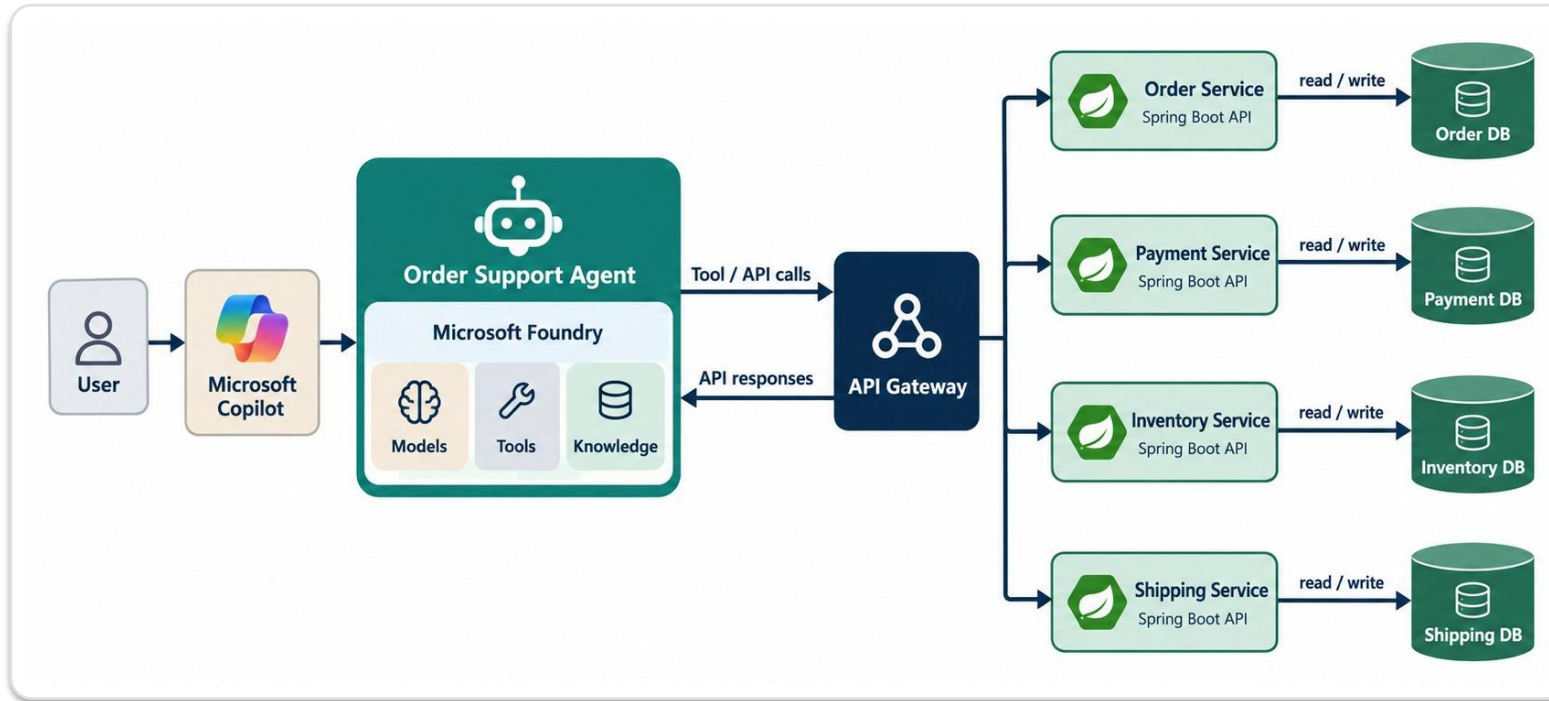
Nothing is cancelled until a person approves.

SAME RULES Order Service still checks the order's state.

KEY TAKEAWAY Expose agent actions as narrow, idempotent, audited requests — the service keeps the final say.

AGENTIC WORKFLOW WITH SPRING BOOT MICROSERVICES

Agent → API gateway → Order, Payment, Inventory and Shipping services.



BAD DESIGN

- ✗ Direct database access
- ✗ A shared super-user token
- ✗ Calling internal-only endpoints

Better design

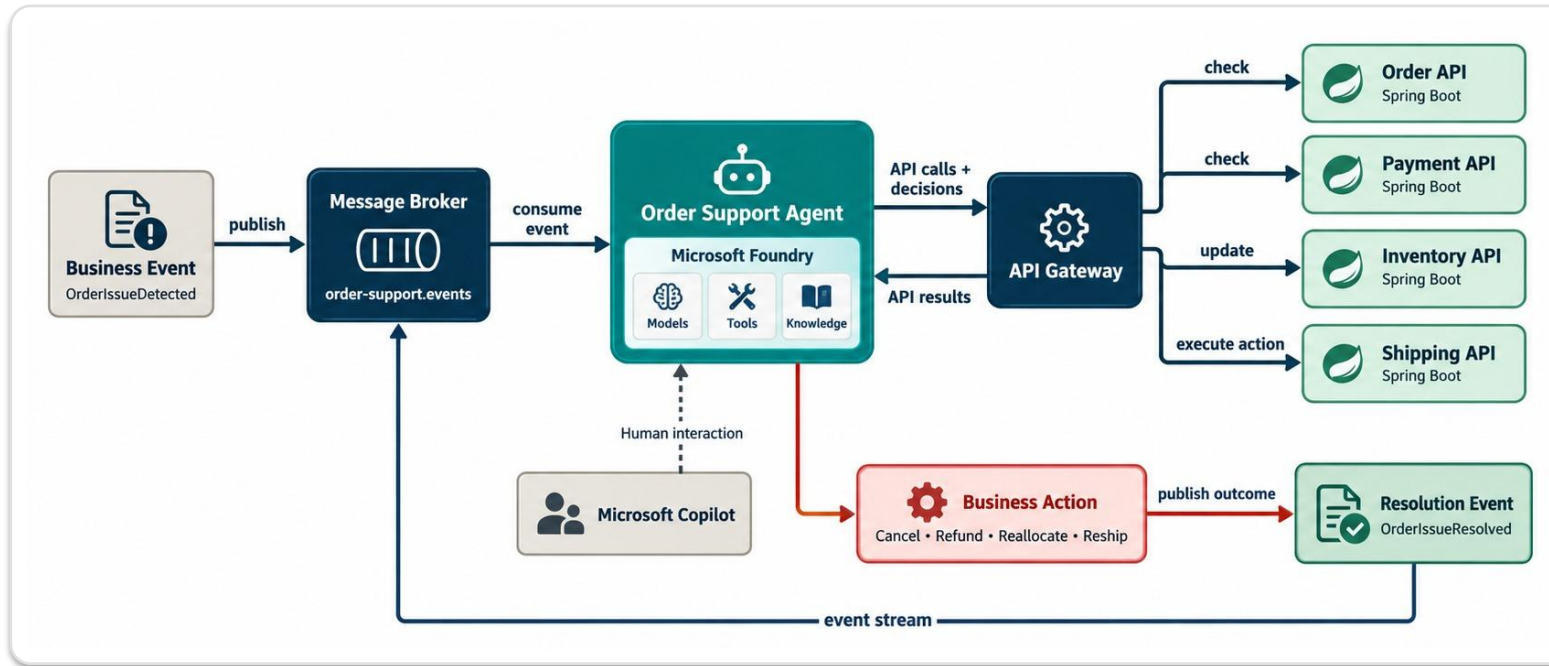
Through the gateway, with the user's or a scoped workload identity.

WHY Validation, security and audit already live in the services.

KEY TAKEAWAY Agentic AI must respect the service boundaries established by the microservices.

EVENT-DRIVEN AGENTIC WORKFLOWS

Events trigger agents; agents invoke services and publish outcomes.



An example

PaymentFailed

- Incident agent investigates
- calls Payment + Order APIs
- publishes IssueTriaged

Idempotent consumer

The agent may receive the same event twice.

MODULE 4 Events, DLQs and correlation IDs still apply.

KEY TAKEAWAY Events can start agents, and agents can publish events — powering autonomous operations.

EXERCISE: INVESTIGATE A FAILED PAYMENT

Design an assistant that investigates — and drafts, but doesn't send.

PRACTICE EXERCISE

12 MIN

SCENARIO

An AI assistant is asked to investigate why a customer's payment failed and draft a response for the support agent.

YOUR TASKS

- 1 Which microservice APIs should it call?
- 2 Which data must be masked?
- 3 Should it send the response automatically?
- 4 What is logged — and what if Transaction Service is down?

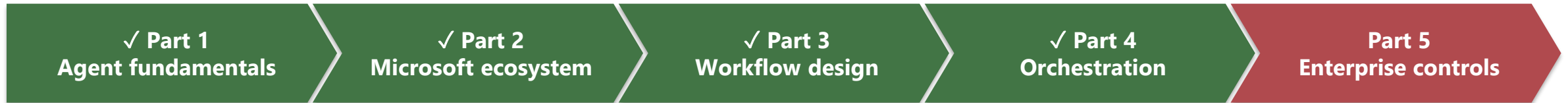
EXPECTED OUTCOME

- ✓ Read: payment, transaction and order status
- ✓ Mask account numbers and personal data
- ✓ No — a person reviews and sends
- ✓ Log tools, IDs, correlation and outcome
- ✓ Service down → say so, don't guess

KEY TAKEAWAY Read-only tools, masked data, a human sender, full audit — and honest answers when a service is down.

PART 5: ENTERPRISE CONTROLS

Identity, policy, approval, reliability and audit for every agent action.



SO FAR

- ✓ Agents call services through the gateway
- ✓ Tool endpoints are narrow and idempotent
- ✓ Events can start agent workflows

NEXT QUESTION

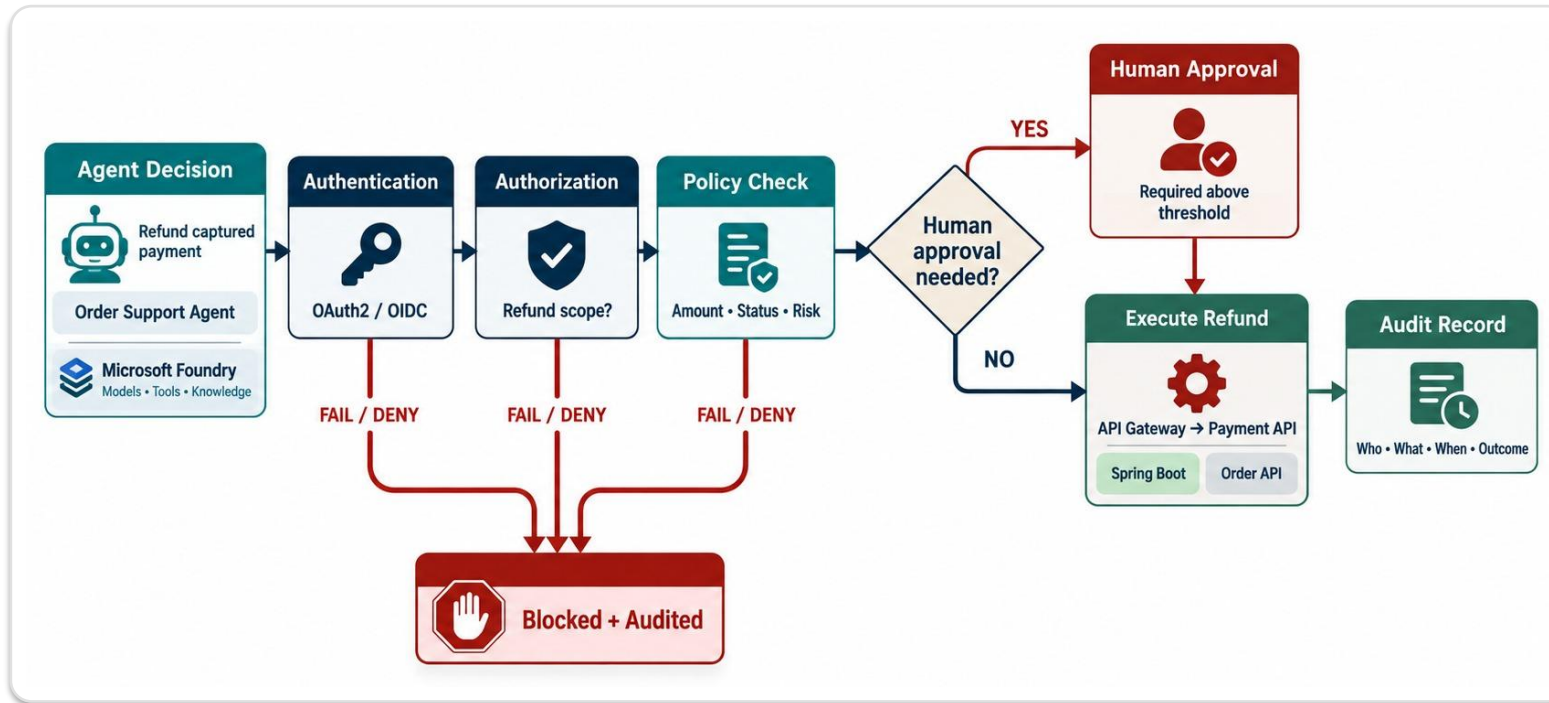
How do we trust an agent in a regulated enterprise?

IN THIS PART

- 1 Guardrails and human approval — in code
- 2 Model versus policy — and Exercise 5.1
- 3 Reliability and observability
- 4 Enterprise use cases
- 5 The complete architecture

GUARDRAILS, SECURITY, AND HUMAN APPROVAL

Authorization, data boundaries, approval gates and auditability.



Identity

The agent acts as the user — or a scoped workload identity.

Policy

Deterministic rules decide what's allowed; the model doesn't.

Approval

High-risk actions wait for a person with the right role.

AUDIT Who asked, what ran, who approved, the outcome.

KEY TAKEAWAY Every agent action passes identity, authorization, policy and — when risky — human approval.

MODEL SCORES, POLICY DECIDES, PEOPLE APPROVE

Lab 5's boundary in code: a model failure becomes HOLD, never APPROVE.

ModelClient.java

```
public ModelResult score(Transaction tx) {
    try {
        return modelApi.post()
            .uri("/v1/score")
            .body(ScoreRequest.from(tx))      // no PII
            .retrieve()
            .body(ModelResult.class);
    } catch (RestClientException e) {        // timeout or 5xx
        log.warn("Model unavailable transactionId={}", tx.id());
        return ModelResult.unavailable();
    }
}
```

✓ AssessmentService.java

```
@Transactional
public Assessment assess(TransactionSubmitted event) {
    ModelResult model = modelClient.score(event.transaction());
    Decision decision = policyEngine.decide(model, event.amount());

    Assessment a = Assessment.of(event, model, decision);
    if (decision.isHold())
        a.markPendingReview();    // needs risk.write to resolve
    return assessments.save(a);
}
```

Model

Returns a score and metadata — never a disposition.

Policy

Deterministic Java decides APPROVE, HOLD or DECLINE.

People

A reviewer with risk.write resolves every HOLD.

KEY TAKEAWAY Keep the model advisory: deterministic policy decides, failures fail safe, and people review holds.

EXERCISE 5.1 — MAP MODEL CALL → POLICY → DISPOSITION

Day 5 checkpoint A: clarity on the governance boundary.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-05/exercises/exercise-5.1-model-policy-disposition.md

SCENARIO

Lab 5's Risk Assessment Service consumes TransactionSubmitted, calls a model on OpenShift AI, and records a disposition.

Scope: Account, Transaction and Risk Assessment only.

YOUR TASKS

- 1 The model returns a score and metadata — it does not approve, hold or decline
- 2 PolicyEngine is deterministic Java; HOLD routes to a human with risk.write
- 3 Timeout or 5xx → HOLD
MODEL_UNAVAILABLE — never auto-approve
- 4 Workbenches and KServe stay lecture — call HTTP :8090 or MD287_MODEL_ROUTE

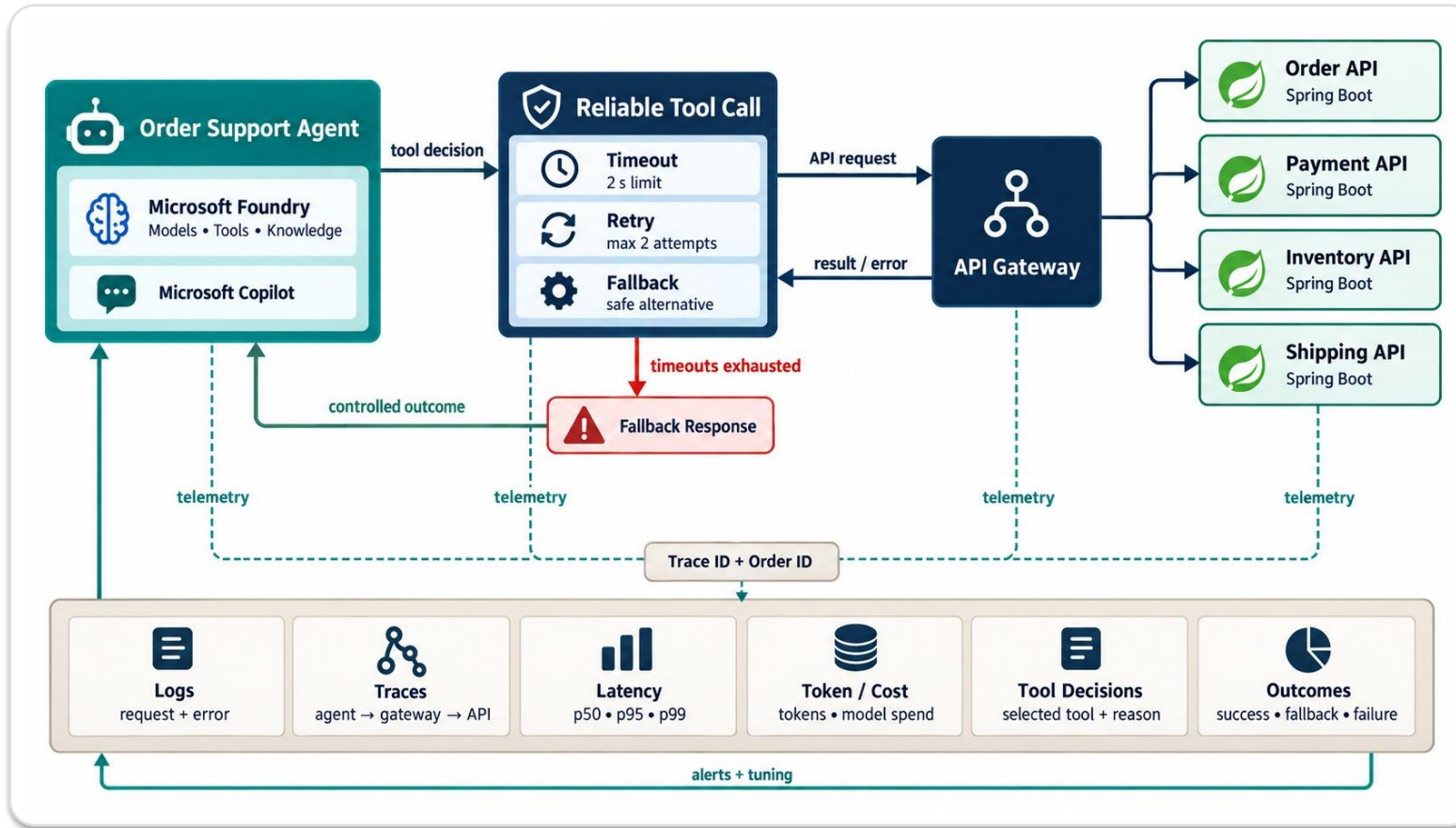
EXPECTED OUTCOME

- ✓ A three-box map
- ✓ Model (score) → Policy (disposition)
- ✓ → Human review (HOLD only)
- ✓ Model failure is a HOLD

KEY TAKEAWAY A three-box map: Model (score) → Policy (disposition) → Human review (HOLD only).

RELIABILITY AND OBSERVABILITY FOR AI AGENTS

Timeouts, retries, tool failures, tracing and token/cost monitoring.



An agent audit record

```
{
  "agentRunId": "run-7f31",
  "tool": "request_order_cancellation",
  "caller": "user-2231",
  "correlationId": "c-9a8e",
  "outcome": "PENDING_APPROVAL",
  "tokens": 1840, "latencyMs": 2310
}
```

WATCH

Tool errors

Steps per run

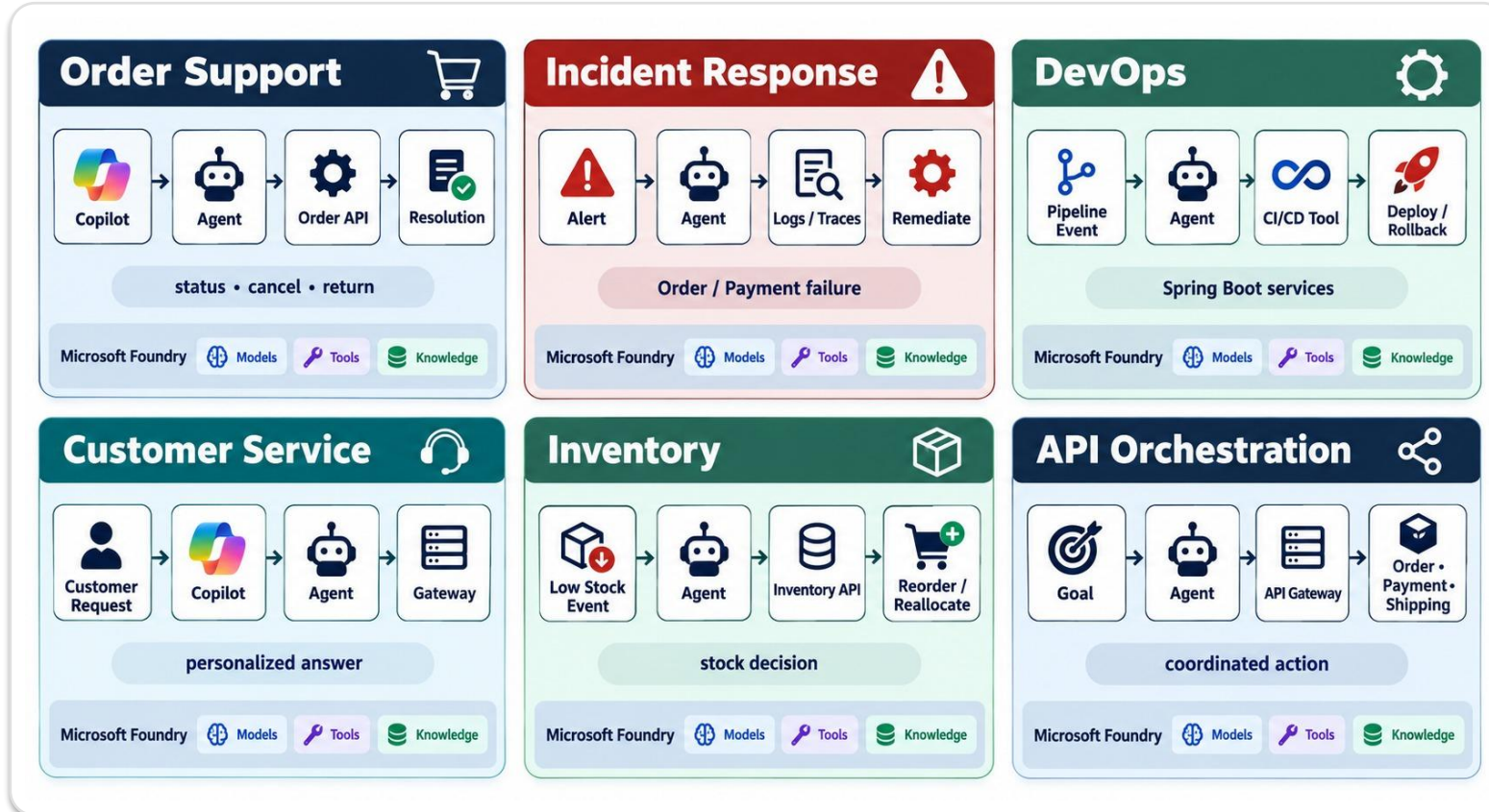
Token cost

Escalations

KEY TAKEAWAY Make the whole reasoning-and-action path resilient and observable.

ENTERPRISE USE CASES IN MICROSERVICES ENVIRONMENTS

Operations, support, incident handling, order workflows and developer automation.



GOOD FITS

- ✓ The goal spans several services
- ✓ Steps are repetitive but vary
- ✓ Tools and data are well governed

POOR FITS

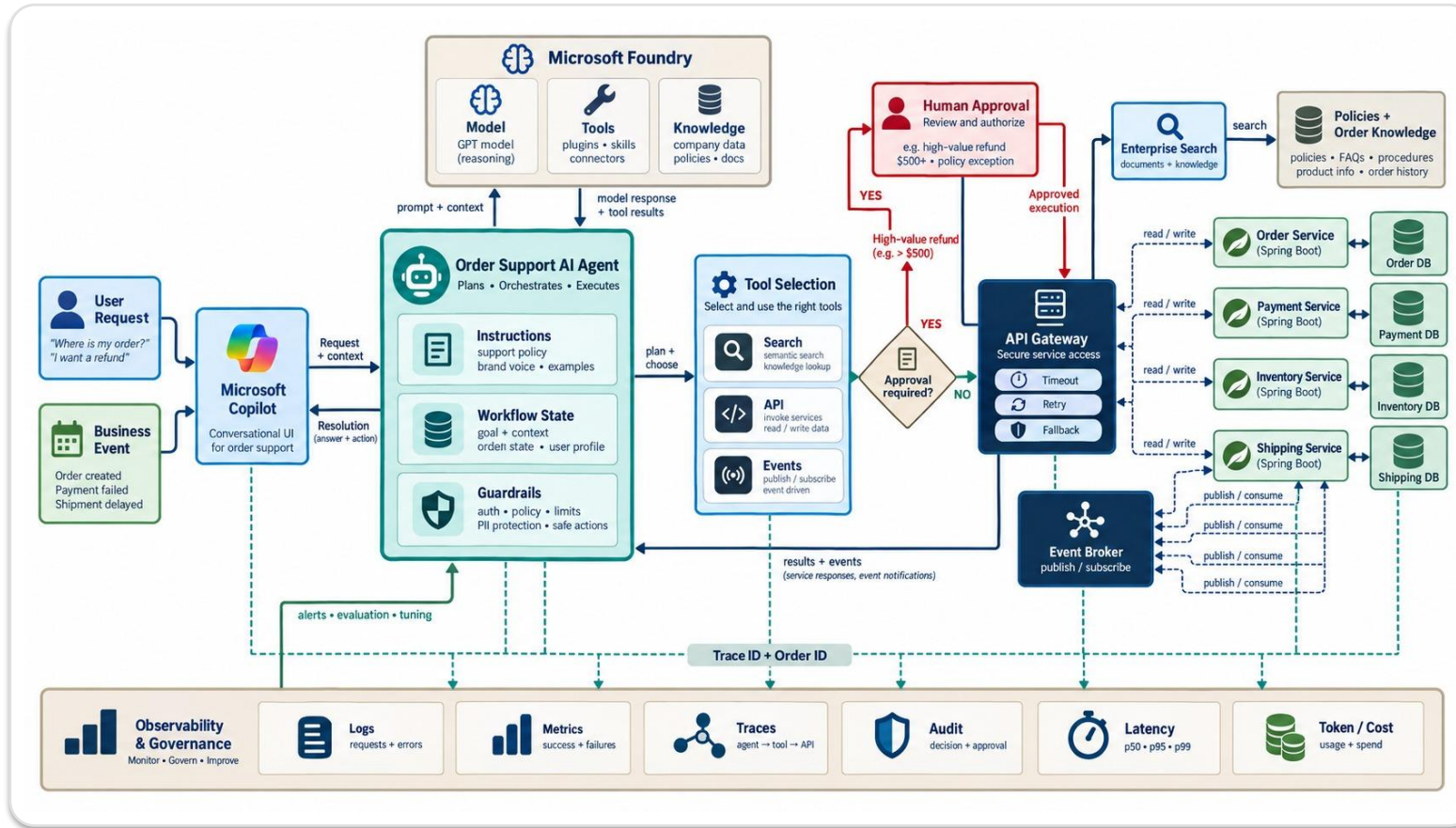
- ✗ A fixed rule is enough
- ✗ No clear owner or audit

START read-only, then add approved actions.

KEY TAKEAWAY Agentic AI adds the most value where a goal spans several enterprise services.

END-TO-END AGENTIC AI MICROSERVICES ARCHITECTURE

User/Copilot → agent → Microsoft AI → tools/APIs → microservices → observability.



EVERY PART OF THE MODULE

- ✓ Agent with instructions, state, guardrails
- ✓ Foundry model requests tools
- ✓ Tools reach services via the gateway
- ✓ Services and policy keep authority
- ✓ Approval, audit, traces and cost

REMEMBER Agents orchestrate above the services.

KEY TAKEAWAY Agents orchestrate above enterprise capabilities — never bypassing APIs, data ownership or controls.

AGENTIC AI MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that make agents unsafe, costly or unreliable.

MISCONCEPTION

REALITY



ANTI-PATTERNS TO AVOID

- ✗ Direct database access for agents
- ✗ One super-user token for all tools
- ✗ No step, time or cost limits
- ✗ Sending customer data unmasked
- ✗ Auto-executing refunds or freezes
- ✗ No tracing of tool decisions

REMEMBER Controlled autonomy over governed capabilities.

KEY TAKEAWAY Useful agents have narrow tools, clear limits, human approval for risk and full audit.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is agentic AI?

2 How does an agent differ from a chatbot?

3 Why should agent tools be limited?

4 What is a human-in-the-loop workflow?

5 Name two Microsoft services for AI workflows.

6 Why is API orchestration useful for agents?

7 What data shouldn't be sent to AI services?

8 Why are audit logs important for agents?

9 What is a stop condition?

10 Why must high-risk banking actions be approved?

MODULE 10 SUMMARY AND WHAT'S NEXT

From prompt-response AI to controlled agents acting on microservices.

YOU CAN NOW

- ✓ Explain agents, their components and the execution loop
- ✓ Map agent needs to Copilot Studio, Foundry and grounding
- ✓ Design narrow tools, bounded loops and stop conditions
- ✓ Orchestrate Spring Boot services through safe tool endpoints
- ✓ Keep policy, approval and audit around every action

NEXT: MODULE 11 — MCP AND MICROSERVICES

- 1 How do agents discover tools in a standard way?
- 2 How do we expose Spring Boot APIs as MCP tools?
- 3 How do we secure and scale MCP servers?

BRING ALONG The tools you designed today — Module 11 standardizes them with MCP.

KEY TAKEAWAY Module 11 gives agents one standard, secure way to reach microservice capabilities.